Waybill — a courier routing engine that proves its own answers

The assignment says TSP, Greedy, and Branch & Bound. Couriers do not solve TSP. This is the plan for building what they actually solve, on real roads, and measuring exactly how far from optimal every method lands.

TRUE PROBLEM	SOLVERS	DISTANCES	VALIDATION	PHASES
CVRPTW	9 across 2 classes	Self-hosted OSRM	TSPLIB known optima	8, each shippable

00 In plain terms

> No jargon in this section. If you can explain the project this way to someone who has never written code, you understand it well enough to defend it.

What is the actual job?

A delivery company has a warehouse and 150 parcels to deliver today. It has 6 vans. Somebody has to decide two things: **which van takes which parcels**, and **what order each driver visits their stops in**.

Get it right and every driver is home by 5pm having burned less fuel. Get it wrong and one van is overloaded, another is half empty, and somebody is still driving at 8pm. That decision is made every single morning by every delivery company on earth. This project builds the thing that makes it.

Why can't a computer just try every option?

Because the number of options doesn't grow steadily — it explodes. Each stop you add multiplies the possibilities by roughly the number of stops you already had.

10 STOPS	15 STOPS	20 STOPS
181,440	**43.6 billion**	**60.8 quadrillion**
possible routes. A laptop checks all of them before you blink.	possible routes. Now about 44 seconds of solid computing.	possible routes. Checking a billion every second: **1.9 years**.

Twenty stops is a light morning for one driver. So "just check everything" stops working almost immediately — and no faster computer rescues you, because adding five more stops multiplies the work by millions again. This is what people mean when they call a problem **hard**: not fiddly, but genuinely beyond brute force forever.

So what do you do instead?

There are two honest answers, and this project builds both so it can compare them.

The quick way — be greedy. Don't plan ahead. Just repeatedly drive to the nearest stop you haven't visited yet. It's like being at a party and always talking to whoever is standing closest. It's instant and the result is reasonable — but because you never planned ahead, you often end up stranded on the wrong side of the room at the end, facing one long miserable drive home. In practice this gives routes about **25% longer than the best possible**.

The careful way — be clever about what you skip. You still can't check 60 quadrillion routes. But you don't have to. Suppose you've already found a route costing 87 kilometres. Now you look at a huge group of routes that all begin with the same bad first move, and you can prove that *even in the luckiest case*, anything in that group costs at least 94. You throw the entire group away without checking a single one of them.

That is **Branch & Bound**, and it's the most elegant idea in the project. You're not searching faster — you're proving that enormous regions of the search can't possibly contain the winner. Done well, it finds the *guaranteed* best route for 50 or 60 stops in seconds, when brute force would still be running when the sun burns out.

Then what makes this project different from every other one?

Most student projects implement one of these and print a route. The route looks plausible, and nobody can tell whether it's good, because there's nothing to compare it to.

This project builds **both kinds**, runs them **on the identical delivery problem**, and measures the difference. So instead of "here is a route," the result is:

> "The greedy method took 0.003 seconds and was 24% worse than perfect. Adding a simple improvement step took it to 4% worse. The exact method proved the perfect answer — but gave up entirely past 19 stops on this laptop."

That's a measurement, not a claim. Anyone can re-run it and check. And we validate against **a public library of problems whose perfect answers are already published and independently verified**, so the accuracy isn't taken on trust.

Why does the map part matter?

Most projects measure distance in a straight line, as if vans could fly. Two addresses might be 2 km apart as the crow flies but a 3.4 km drive, because of one-way streets, a river, and a railway crossing. Worse, driving A→B is often a different distance than B→A, because of one-ways.

So this project runs a real map engine on real street data. Distances are true driving distances, and the routes drawn on screen follow actual roads instead of cutting through buildings. It's the difference between a diagram and a tool.

What will it actually look like when it's finished?

A map of a real city. You drop pins where the deliveries are, say how many vans you have and how much each one holds, and optionally give some stops a time window — "this one must be between 9 and 11am." You press solve, and coloured routes appear, one per van, following real streets, each stop labelled with the time the driver should arrive.

Alongside it, a panel that tells you the truth about the answer: which method produced it, how long it took, and how far from perfect it is likely to be. And if a new order comes in halfway through the day, the plan absorbs it without tearing up the stops already completed.

THE FIVE WORDS WORTH KNOWING

TSP

Travelling Salesman Problem. One vehicle, a list of stops, no limits: what's the shortest round trip that visits every stop once? The famous hard problem this project is built on.

GREEDY

Any method that always takes the choice that looks best right now, without considering the consequences later. Fast, simple, and usually pretty good —

but never guaranteed.

BRANCH & BOUND

Split the possibilities into groups ("branch"), then calculate the best conceivable outcome for each group ("bound"). If a group's best conceivable outcome is still worse than an answer you already have, discard the whole group unopened.

OPTIMALITY GAP

How much worse a fast answer is than the perfect one, as a percentage. The single most important number in the project — because it turns "this seems fine" into evidence.

CVRPTW

The realistic version of the problem: multiple vehicles, each with a capacity limit, and stops with delivery time windows. Ugly name, but it's the problem actual couriers have.

---

01 The problem the title gets wrong

> TSP is the kernel. It is not the system. Building the kernel and calling it a courier system is the single reason these projects read as coursework.

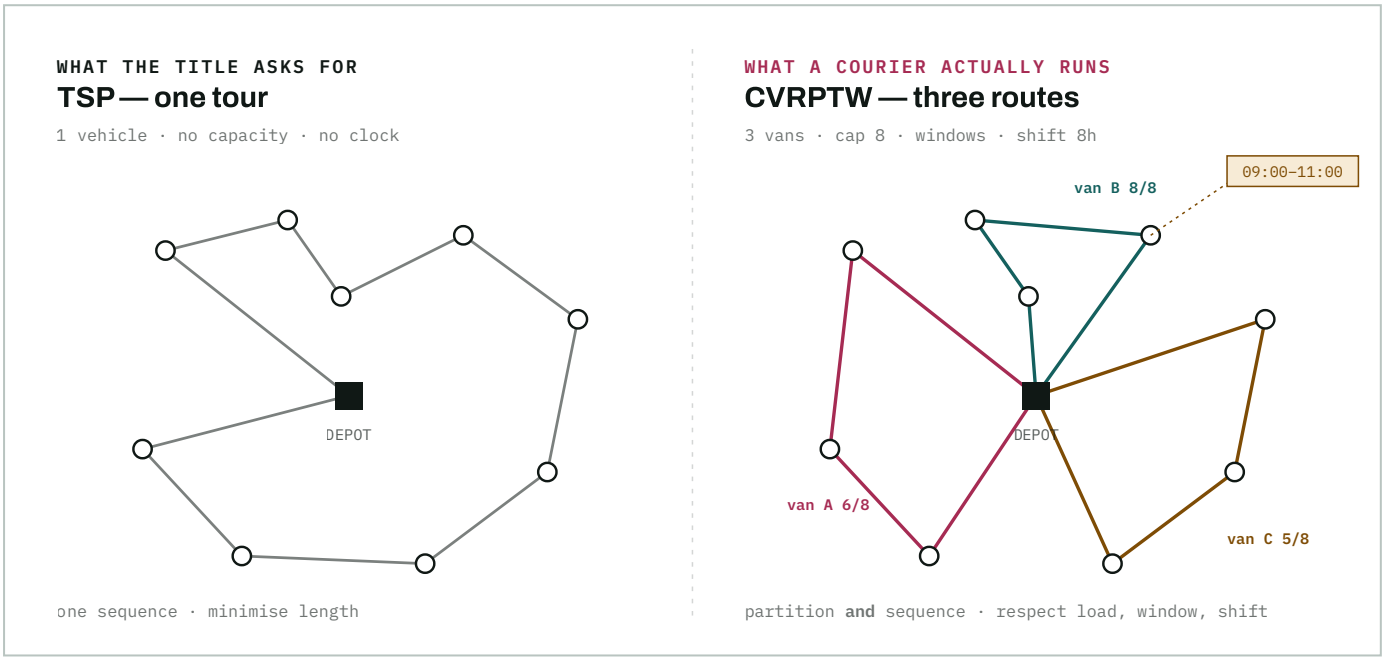
The **Travelling Salesman Problem** asks for the minimum-cost Hamiltonian cycle over `n` nodes — visit every stop exactly once, return to the start, minimise total cost. It is NP-hard. A symmetric instance has `(n−1)!/2` distinct tours, so `n = 20` is already about 6×10^{16} of them. That difficulty is real and worth respecting.

But a courier fleet violates every simplifying assumption TSP makes:

- **Multiple vehicles.** You dispatch a fleet, not a salesman. Deciding which van takes which stops is a partitioning problem that sits on top of the sequencing problem — and it dominates the cost.
- **Capacity.** A van holds a finite weight, volume, and piece count. A tour that is optimal in distance is worthless if it needs 1.4 vans.
- **Time windows.** "Deliver between 09:00 and 11:00" turns a pure ordering problem into a scheduling problem. Arriving early means waiting; arriving late means failing.

- **Service time.** Each stop consumes minutes at the door — parking, lift, signature. Often larger than the driving time between adjacent urban stops.
- **Asymmetry.** On a real road network `d(A,B) ≠ d(B,A)`. One-ways, turn restrictions, divided highways. Every textbook symmetric-matrix assumption breaks.
- **Shift limits.** Drivers have legal maximum hours and mandated breaks.

The correct name for this is **CVRPTW** — Capacitated Vehicle Routing Problem with Time Windows. TSP is the special case where you have one vehicle, infinite capacity, and no clock.



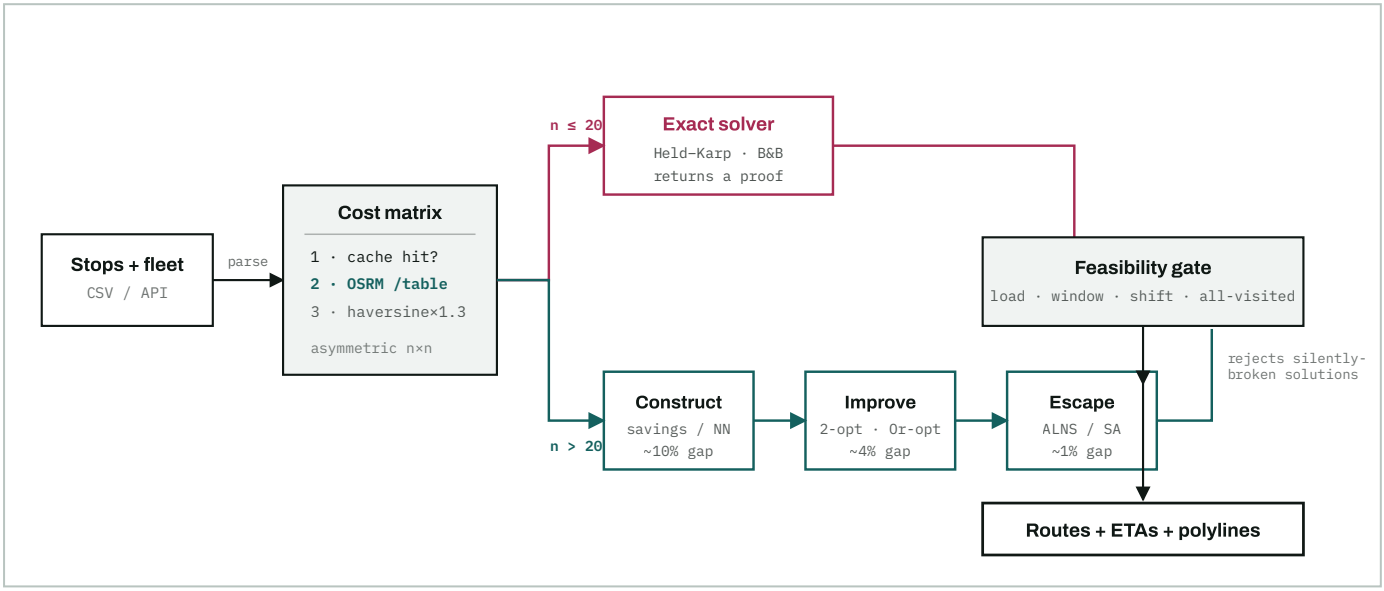
Identical nine stops, two different problems. The left panel optimises a sequence. The right panel must first decide which van gets which stops, then sequence each one, then check every arrival against a clock and every load against a limit. The partitioning decision on the right is where most of the cost — and all of the difficulty — actually lives.

THE FRAMING THAT WINS

Do not quietly ignore the assignment. Implement TSP with Greedy and Branch & Bound exactly as specified, prove they work — then show, with your own measurements, that the TSP formulation cannot express a real delivery day, and extend it. You are then answering the assignment *and* critiquing it with evidence. That is a much stronger position than either alone.

02 How the system works, end to end

A solve is a pipeline. Addresses become coordinates, coordinates become a cost matrix, the matrix feeds a solver, and the solver's output is checked against every constraint before it is allowed out. The branch in the middle is the interesting part: below a size threshold the system returns a **provably optimal** answer; above it, a heuristic answer with a measured quality estimate.



One pipeline, two lanes. Small instances take the upper lane and come back with a certificate of optimality. Everything else takes the construct → improve → escape chain, where each stage measurably narrows the gap. Both lanes pass the same feasibility gate, so a heuristic can never return a solution that quietly overloads a van or misses a window.

> **WHY THE GATE IS SEPARATE**
>
> Feasibility checking lives outside every solver, and every solver's output goes through it. This sounds redundant — solvers are supposed to respect constraints. It catches the bug class that otherwise eats a week: a local-search move that improves distance while silently pushing arrival past a window. An independent checker turns that from a mystery into a failing test.

03 The solver ladder

Nine solvers, two classes. `EXACT` methods return the true optimum and a proof; they die at a size wall. `HEURISTIC` methods scale indefinitely and return an answer of unknown quality — which is precisely what section 05 exists to measure.

Gap figures below are typical published ranges for random Euclidean instances. **Your job is to reproduce them on your own machine, with your own code** — not to cite them. If your numbers differ, that difference is a result worth explaining.

METHOD	CLASS	COMPLEXITY	PRACTICAL N	TYPICAL GAP	ROLE IN THE SYSTEM
Brute force	EXACT	$O(n!)$	≤ 10	0%	Ground truth for unit tests
Held–Karp DP	EXACT	$O(n^2 \cdot 2^n)$ time, $O(n \cdot 2^n)$ memory	≤ 20	0%	Exact reference; dies on *memory*, not time
B&B — reduced cost	EXACT	exponential, data-dependent	15 – 25	0%	The assignment's named technique
B&B — 1-tree bound	EXACT	exponential, far better pruning	40 – 70	0%	Shows you understand *bounds*, not just search
Nearest neighbour	HEUR	$O(n^2)$	any	~25%	Instant seed; the baseline to beat
Clarke–Wright savings	HEUR	$O(n^2 \log n)$	any	~10%	Handles multi-vehicle + capacity natively
Cheapest insertion	HEUR	$O(n^2 \log n)$	any	~15%	Powers live insertion of new orders
2-opt + Or-opt	HEUR	$O(n^2)$ per pass	any	~4%	Local search; the biggest single win
ALNS / guided local search	HEUR	anytime, budgeted	1000+	0.5 – 2%	Production solver

Read the table as a narrative, because that narrative *is* the report: greedy gets you 75% of the way in microseconds, local search closes most of the remaining gap for almost nothing, and the last 2% costs more compute than everything before it combined. Exact methods are simultaneously the most rigorous and the least useful — and being able to say precisely where that crossover sits, with your own graph, is the deliverable.

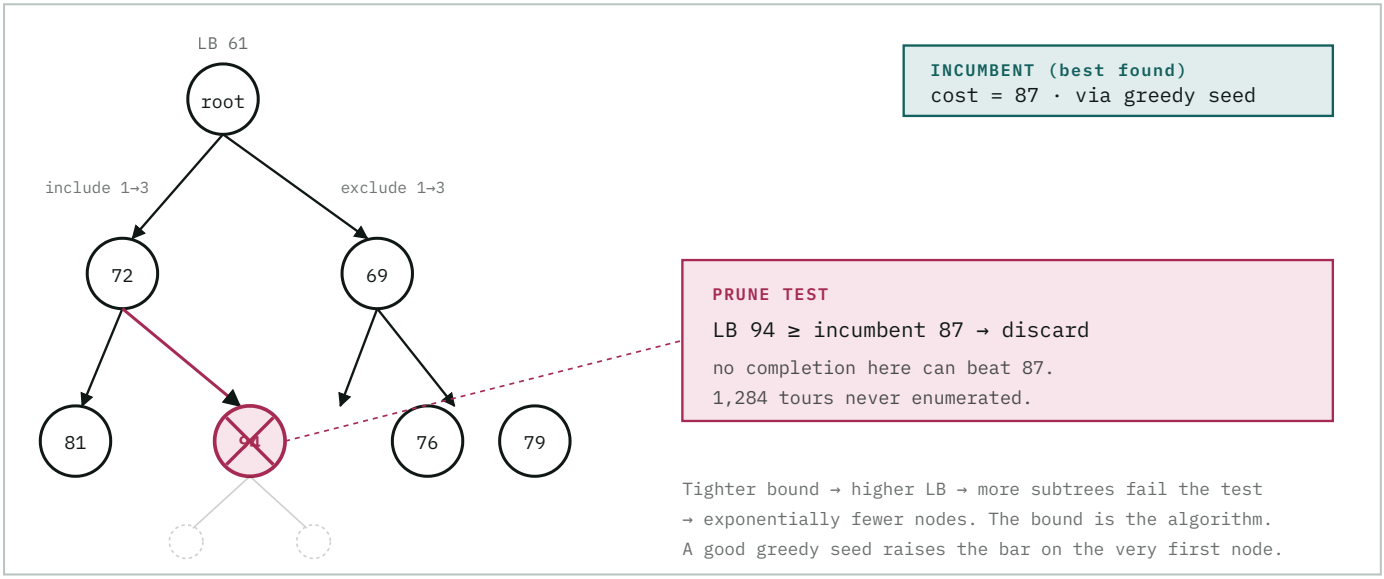
Clarke–Wright deserves special attention

The savings algorithm is the one classical heuristic that is **natively a fleet algorithm**. It starts with one dedicated out-and-back route per stop, computes the saving from merging any two routes — `s(i,j) = d(depot,i) + d(depot,j) − d(i,j)` — then merges greedily in descending saving order, skipping any merge that would breach capacity. It answers the partitioning question and the sequencing question in one pass, and it is what real dispatch systems ran on for decades. Implement it; it is the backbone of your multi-vehicle story.

04 Branch & Bound, done properly

> Branch & Bound is not a search algorithm with a bound bolted on. It is a bound with a search wrapped around it — and the bound is where all the engineering lives.

The mechanism: maintain the best complete tour found so far (the **incumbent**, an upper bound). For each partial tour, compute a **lower bound** on the best possible completion. If that lower bound is already ≥ the incumbent, no completion of this partial tour can beat what you have — discard the entire subtree without enumerating it. Every pruned node is a factorial number of tours you never touch.



One prune, and a subtree vanishes. The crossed node's lower bound of 94 already exceeds the best known tour of 87, so every tour beneath it is eliminated without being built. This is the entire value proposition of B&B — and it explains why a better bound function beats a faster search loop every time.

Implement three bounds, not one

This is the highest-value technical decision in the project, because it turns "I wrote B&B" into "I measured why bound quality dominates search strategy."

- **Row/column reduction** (Little et al., 1963). The textbook bound. Reduce each row then each column of the cost matrix by its minimum; the accumulated reduction is a valid lower bound. Easy, weak, and the one everyone submits.
- **Assignment-problem relaxation.** Drop the "single connected tour" requirement and solve the resulting assignment problem with the Hungarian algorithm in $O(n^3)$. Much tighter than reduction, because it is an exact solve of a genuine relaxation.

- **1-tree / Held–Karp bound with subgradient ascent.** A minimum spanning tree over nodes `2…n` plus the two cheapest edges at node 1 is a valid relaxation of a tour. Then iteratively perturb node potentials to tighten it. This routinely lands within 1% of the true optimum and is what makes B&B viable past n = 40.

INSTRUMENT EVERYTHING

Every B&B run must emit: nodes created, nodes explored, nodes pruned, incumbent value over time, peak frontier size, wall time. Without these you have an implementation. With them you have graphs showing nodes-explored climbing exponentially with `n`, three bound functions separating cleanly, and the precise `n` where your laptop gives up. That is the result.

> SEARCH STRATEGY MATTERS LESS THAN YOU THINK
>
> Best-first search finds the optimum with fewest nodes but its frontier grows until memory dies. Depth-first uses almost no memory and finds good incumbents early, which improves pruning immediately. Use **DFS with a greedy-seeded incumbent** as the default, and offer best-first as a comparison mode. Then measure both and report the difference — including the memory blow-up, which is the more interesting finding.

---

05 Proving the gap — the section that carries the project

> Anyone can implement nine algorithms. Almost nobody measures them honestly against a known optimum. Do that, and the project stops being coursework and becomes a small experimental result.

Three tiers of evidence

1. **Self-validation.** Generate random instances with `n ≤ 12`. Solve with brute force — indisputably optimal. Run every other solver on the identical instance. Any exact solver disagreeing with brute force is a bug, and this catches it instantly. This is a test suite, not a benchmark.
2. **Gap measurement.** For `n` from 5 to 20, generate 50 random instances per size. Solve exactly with Held–Karp, then with each heuristic. Report mean and standard deviation of `(heuristic − optimal) / optimal`. You now have error bars, which almost no student project has.
3. **External validation.** Run against TSPLIB — the standard benchmark library whose optimal tour lengths are published and peer-verified. Report your percentage above the published optimum for `berlin52`, `eil76`, `kroA100`, `ch150`, `pr226`. Anyone can check your numbers. That is the point.

The four plots that make the report

- **Gap vs n** — one line per heuristic, error bars from the 50 runs. Shows the quality ladder collapsing onto the optimum.
- **Runtime vs n**, log scale — exact methods curving vertically off the chart while heuristics stay near-flat. This single plot is the entire argument for heuristics.
- **B&B nodes explored vs n**, one line per bound function. Shows bound quality buying you orders of magnitude.
- **Anytime quality curve** — solution cost against wall-clock for ALNS on a large instance, with the greedy start and the best-known value drawn as horizontals. Shows the metaheuristic converging in real time.

> USE OR-TOOLS AS A RULER, NEVER AS THE ENGINE
>
> Google OR-Tools will beat your solver. Install it, run it as a reference column in the benchmark table, and report the difference honestly — "our ALNS lands 1.8% above optimal on kroA100; OR-Tools reaches 0.3%" is a *strong* sentence, because it proves you measured against a serious baseline. What destroys credibility is wrapping OR-Tools and presenting it as your implementation. Keep it strictly in `bench/`, never in `core/`.

---

06 Real roads, not straight lines

The fastest way to make this project stop looking academic: **delete the Euclidean distance function.** Straight-line distance between two urban addresses understates real driving distance by roughly 20–40%, and it is symmetric, which real road networks are not.

You have Docker installed, so the good option is available: **self-host OSRM** (Open Source Routing Machine) against an OpenStreetMap extract of one city or region. Three commands to preprocess, then you own a routing engine with no API keys, no rate limits, and no network dependency.

Two endpoints carry the whole system:

- `/table` — returns the full `n×n` travel-time and distance matrix in one call. This is the solver's input, and OSRM computes it in milliseconds for hundreds of stops.
- `/route` — returns the actual driving geometry as an encoded polyline. This is what makes the map look real: routes that follow streets instead of cutting through buildings.

FALLBACK CHAIN

Build the distance layer as an ordered chain so the system never hard-fails: **persistent cache → OSRM → Haversine × 1.3**. Cache on a hash of the rounded coordinate pair, since the same depot-to-customer legs recur constantly. The Haversine fallback keeps tests fast and hermetic, and keeps the project runnable on a machine with no Docker.

> SIZING WARNING
>
> Take a *city or small-region* OSM extract, not a national one. A metro-area extract preprocesses in a few minutes on a laptop; a whole-country extract can need tens of gigabytes of RAM and hours of CPU. This is the most common way people lose a day on this step.

---

07 The constraints that make it a system

Each constraint below changes what "valid" means, and therefore changes every local-search move. Add them in this order — each one builds on the last.

- **Capacity.** Track weight, volume, and piece count independently; a van fills up on volume long before weight for parcel work. A move is infeasible if any dimension breaches.
- **Time windows.** Each stop has `[earliest, latest]`. Arriving before `earliest` means waiting — legal but wasteful. Arriving after `latest` is infeasible for hard windows, or penalised for soft ones. Support both; soft windows produce far better plans on messy real data.
- **Service time.** Fixed dwell per stop, plus optionally a per-parcel component. In dense urban routes this exceeds driving time.
- **Shift limits.** Route duration from depot departure to depot return, capped. Add mandated breaks if you want to go further.
- **Priority tiers.** Express vs standard, expressed as a penalty weight in the objective rather than a hard constraint — so the solver can make an intelligent trade instead of failing.

The optimisation that makes time windows tractable

This is the detail that separates a working implementation from an unusably slow one, and it is worth stating precisely.

A naive time-window feasibility check on a candidate move re-walks the route to recompute every downstream arrival time — `O(n)` per move. Local search evaluates `O(n²)` moves per pass. You are now at `O(n³)` per improvement pass, and a 200-stop instance takes minutes per iteration.

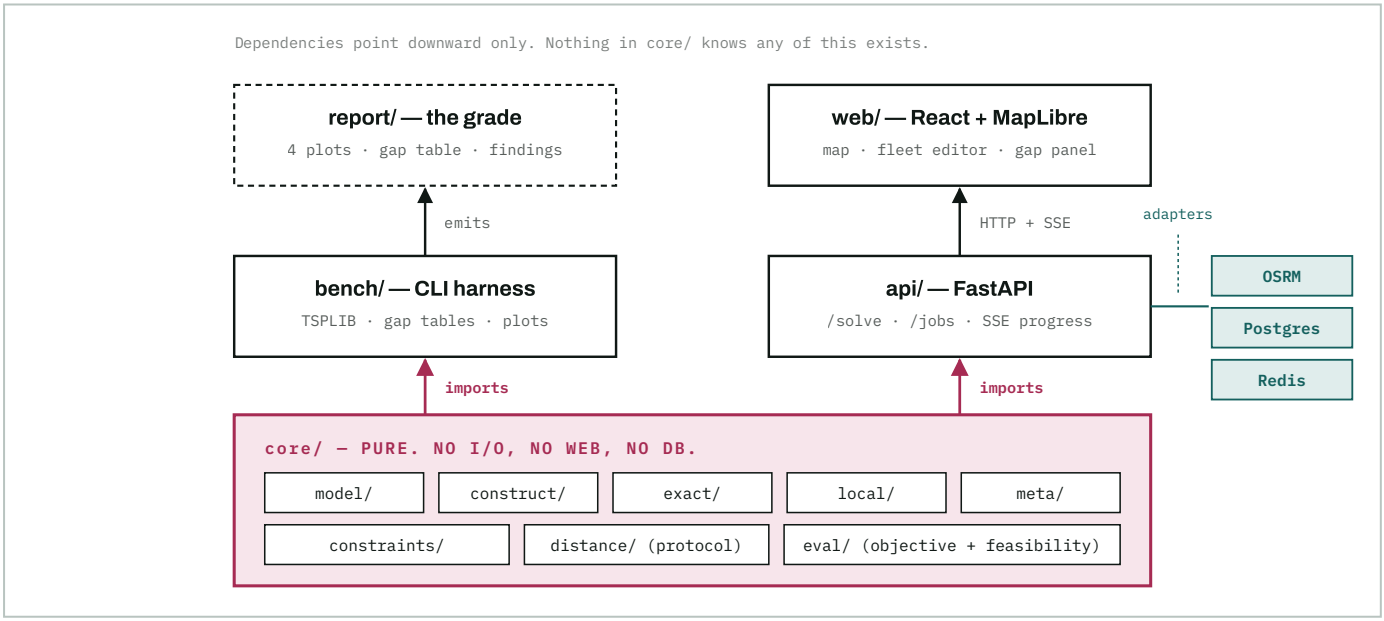
The fix is **forward time slack**: precompute, for each position in the route, how much extra delay can be absorbed before any downstream window is violated. A candidate move's feasibility then reduces to comparing its induced delay against the stored slack — `O(1)`. Recompute slack once per accepted move, not per evaluated move.

> **WHY THIS BELONGS IN THE REPORT**
>
> This is a genuine algorithm-engineering result, measurable and easy to demonstrate: benchmark naive vs slack-based feasibility checking on the same instance and plot both. A two-orders-of-magnitude speedup with no change in solution quality is exactly the kind of finding that shows you understand the difference between an algorithm's complexity and its implementation's.

08 Architecture

> One rule governs the whole layout: `core/` imports nothing from the web, the database, or the network. Everything else depends on it, and nothing it depends on can break a solve.

That rule is what makes the benchmark trustworthy. The code the harness measures is byte-identical to the code the API serves — no "benchmark version," no divergence, no excuses when the numbers disagree.



The core is the product; everything else is a consumer. Because `bench/` and `api/` import the same solver code, a benchmark number is a claim about the deployed system. OSRM, Postgres and Redis attach through adapters at the service layer — `core/` sees only a distance-matrix protocol, which is why the whole solver suite runs offline in unit tests.

Repository layout

```
waybill/
├── core/                  # pure python — zero web/db imports
│   ├── model/               # Instance, Solution, Vehicle, Stop, TimeWindow
│   ├── distance/            # Protocol + haversine, OSRM, cached impls
│   ├── construct/           # nearest_neighbour, savings, insertion
│   ├── exact/               # brute_force, held_karp, branch_bound/
│   │   └── bounds/          # reduction, hungarian, one_tree  ← the differentiator
│   ├── local/               # two_opt, or_opt, relocate, cross_exchange
│   ├── meta/                # simulated_annealing, tabu, alns
│   ├── constraints/         # capacity, time_windows, shift, slack
│   └── eval/                # objective, feasibility, cost model
├── bench/                 # imports core; never imported BY core
│   ├── tsplib/              # loader + published optima
│   ├── runner.py            # sweeps n, repeats, collects stats
│   ├── ortools_ref.py       # reference column only
│   └── plots.py             # the four figures
├── api/                   # FastAPI: thin. no algorithms here.
├── web/                   # React + MapLibre GL
├── infra/                 # docker compose: osrm, postgres, redis
├── data/                  # tsplib instances, sample fleets, osm extract
└── report/                # generated figures + written findings
```

The one interface that holds it together

Every solver — brute force, B&B, ALNS — implements the same protocol: take an `Instance` and a budget, return a `Solution` plus a `SolveReport` carrying node counts, wall time, and an `optimal: bool` flag. The benchmark harness and the API then treat all nine identically, and adding a tenth solver requires changing nothing else. Define this on day one; retrofitting it later is painful.

---

09 Build plan

> Eight phases, ordered so that something real works at the end of each one. The UI is deliberately late — the algorithms are the project, the interface is packaging, and packaging built over a broken core is wasted work.

PHASE 0

Skeleton and ground truth

Repo, dependencies, and the data model: `Instance`, `Solution`, `Vehicle`, `Stop`, `TimeWindow`. The `Solver` protocol. The objective function and the independent feasibility checker. A TSPLIB loader and a random-instance generator. Store all times in integer seconds and all distances in integer metres — float comparison in feasibility checks causes bugs you will not find.

> **SHIPS**
> Load berlin52, score a known tour, assert it matches the published optimum. Tests green.

PHASE 1

Distance layer

The `DistanceProvider` protocol with three implementations: Haversine (fast, hermetic, for tests), OSRM via Docker (`/table` and `/route`), and a caching decorator over either. Geocoding for CSV address ingest.

> **SHIPS**
> A real asymmetric matrix for 50 real addresses in your city, cached to disk.

PHASE 2

Constructive + exact solvers

Nearest neighbour, Clarke–Wright savings, cheapest insertion. Then brute force, Held–Karp DP, and Branch & Bound with the reduced-cost bound. Cross-validate: for `n ≤ 10`, all three exact solvers must return identical costs on every random instance.

> **SHIPS**
> Provably optimal answers up to n≈18, and the assignment's stated requirements are already met.

PHASE 3

Better bounds

Add the Hungarian assignment-relaxation bound and the 1-tree bound with subgradient ascent. Add full instrumentation — nodes created, explored, pruned, peak frontier, incumbent trace. Add DFS and best-first as switchable strategies.

> **SHIPS**
> Nodes-explored curves for three bound functions. Your first genuine finding.

PHASE 4

Local search and metaheuristics

2-opt, Or-opt, relocate, swap, cross-exchange. Then simulated annealing and ALNS with destroy/repair operators. Implement don't-look bits and neighbour lists — without them 2-opt is `O(n²)` per pass and unusable past a few hundred stops.

> **SHIPS**
> Sub-2% gap on kroA100, in under a second.

PHASE 5

Constraints — TSP becomes CVRPTW

Capacity, hard and soft time windows, service time, shift limits. Multi-vehicle throughout. Forward time slack for `O(1)` move feasibility. Every local-search operator must now be constraint-aware.

> **SHIPS**
> A full delivery day: 150 stops, 6 vans, real windows, feasible plan with per-stop ETAs.

PHASE 6

Benchmark harness and report

The sweep runner, the TSPLIB comparison table, the OR-Tools reference column, and the four plots. Write the findings — including the ones that went against expectation. **This phase is where the grade actually lives.**

> SHIPS
> `python -m bench.runner --full` regenerates every figure and table from scratch.

PHASE 7

API and map interface

FastAPI with `/solve`, job management, and server-sent events streaming solver progress so the map animates as the tour improves. React + MapLibre with real OSRM polylines, a drag-to-add-stop editor, a fleet configurator, and an explainability panel showing which solver ran, how long it took, and the measured gap.

> SHIPS
> A live demo: drop pins on a real map, watch routes converge, read the gap.

PHASE 8

Dynamic re-planning

The feature that makes it a system rather than a batch job: insert a new order into a plan already in progress via cheapest insertion plus a local repair, handle a failed delivery, re-optimise only the affected routes while frozen stops stay fixed. Optionally, time-dependent travel matrices for rush-hour traffic.

> SHIPS
> An order arrives mid-day; the plan absorbs it in under a second without disrupting completed stops.

10 Risks and known traps

- **This is genuinely three projects.** A solver library, a backend service, and a mapping frontend. The phase structure is the mitigation — if you run out of time after Phase 6, you still have a complete, defensible, well-measured algorithms project. Never

start Phase 7 before Phase 6 is finished.

- **Floating-point feasibility.** A time window checked with floats will admit an arrival 0.0001 seconds late on one run and reject it on the next. Integer seconds and integer metres internally, formatted only at the boundary.
- **Local search without neighbour lists.** Evaluating all `O(n²)` 2-opt moves every pass makes anything past 300 stops crawl. Restrict candidate moves to each node's 10 nearest neighbours and use don't-look bits.
- **Best-first B&B memory.** The frontier grows exponentially and the process dies without a useful error. Cap frontier size, or default to DFS.
- **OSM extract size.** Metro-area extract, not national. See section 06.
- **Benchmarking noise.** One run per configuration proves nothing. Repeat, report medians with interquartile ranges, fix random seeds so results reproduce, and pin the seed in the report.
- **Comparing across different distance functions.** A gap number is only meaningful when every solver saw the identical matrix. Build the matrix once per instance and pass it to all of them.

---

11 Definition of done

The bar to hold this to — every item verifiable by someone else on their own machine:

- Every exact solver agrees with brute force on 1,000 random instances with `n ≤ 10`.
- TSPLIB results are reported as percentage above published optimum, for at least five named instances.
- The B&B size wall is stated as a specific number with the hardware it was measured on.
- Three bound functions are compared on nodes explored, not just on wall time.
- Real road distances from a self-hosted routing engine, with an asymmetric matrix.
- A 150-stop, 6-vehicle instance with time windows solves to a feasible plan in under 10 seconds.
- All four figures regenerate from one command, from raw code, with a fixed seed.
- An OR-Tools reference column appears in the results table, and your solver is honestly compared against it.
- The map shows routes following actual streets.
- The written findings include at least one result that contradicted your initial expectation.

THE SENTENCE TO AIM FOR

"Branch & Bound with a 1-tree bound solved 60-stop instances to proven optimality in 4.2 seconds, where the textbook reduced-cost bound could not finish 30 stops in ten minutes — a 2,100× reduction in nodes explored from changing nothing but the bound function." That sentence is the project. Everything in this document exists to let you write it truthfully.
